Problem set #10: Monte Carlo

Milica Cudina

2026-10-12

Problem #1: Simple gambling

Consider a game in which a spinner is spun. The spinner is equally likely to land on *yellow*, *red*, or *blue*. If the spinner lands on *yellow*, the player loses $1. If the spinner lands on *red*, the player wins $1. If the spinner lands on *blue*, no money changes hands.

Use *Monte Carlo* to estimate the expected winnings based on 10,000 runs. Does the result agree with your theoretical expected value?

Now, complete 1,000 batches of 10,000 runs. What does the histogram of the averages of the 1,000 batches look like? What is their mean? What is their variance? What is their standard deviation?

Solution: Let's encode "yellow" as -1, red as 1, and blue as 0.

```
spinner=c(-1, 0, 1)
#for replicability
set.seed(1)
(one.run=sample(spinner, 1))
```

```
## [1] -1
```

Now, we do *Monte Carlo*. If we want multiple draws, what do we do?

```
n.sim=10^4

(many.runs=sample(spinner, n.sim, replace=TRUE))
```

```
##   [1]  1 -1  0 -1  1  1  0  0  1  1 -1 -1 -1  0  0  0  0  1 -1  1 -1 -1 -1 -1  0
##  [26] -1 -1  0  0  0 -1  1 -1  1  0  0  0  0  1  0 -1  1  0 -1 -1  1  0  0  1  1
##  [51]  0  0  0  0 -1  0  0  0  0 -1  1  1  0  1  1  0  1  1 -1 -1 -1 -1  1  0  1
##  [76] -1 -1  0 -1 -1 -1 -1  1  0 -1 -1  1  1  1  0  0  0  1  0  0  1  1  1 -1  0
##  [ reached 'max' / getOption("max.print") -- omitted 9900 entries ]
```

Now, we take the arithmetic average of the simulated values to get our *Monte Carlo* estimate of the expectation.

```
(mc.exp=mean(many.runs))
```

```
## [1] -0.0139
```

This is "close to" 0. I am happy!

For the second part of the problem, we define a function in `R` which calculates a single *Monte Carlo* estimate.

```
spin.mc<-function(){
  mean(sample(spinner, n.sim, replace=TRUE))
}
```

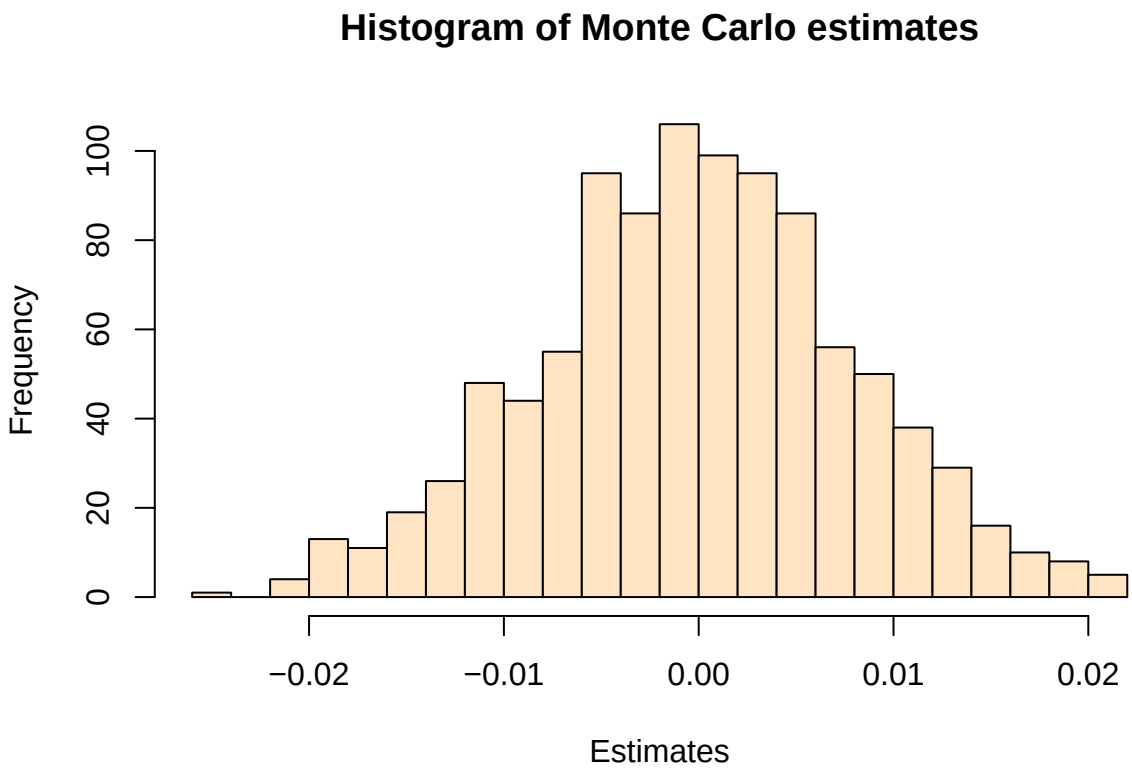
```
#spin.mc()
```

Next, we execute 1000 batches of the above function.

```
n.batches=1000

all.batches=replicate(n.batches, spin.mc())

hist(all.batches,
     main="Histogram of Monte Carlo estimates",
     xlab="Estimates",
     col="bisque",
     breaks=30)
```



The CLT kicked in, we have a normal-distribution-looking histogram.

```
#mean (accuracy)
mean(all.batches)
```

```
## [1] -0.0001457
```

```
#variance (precision)
var(all.batches)
```

```
## [1] 0.00006500531
```

```
#compare to the theoretical variance
(2/3)/10000
```

```
## [1] 0.00006666667
```

```
#standard deviation
sd(all.batches)
```

```
## [1] 0.008062587
```

```
#compare to the theoretical sd
sqrt((2/3)/10000)
```

```
## [1] 0.008164966
```

Problem #2: Gambling

Consider the following game:

- if a fair die lands on an odd number, the player loses one dollar;
- if a fair die lands on an even number which is not a perfect square, no money changes hands;
- if a fair die lands on an even number which is also a perfect square, the player wins nine dollars.

What is the theoretical expectation of the amount won in this game?

Use *Monte Carlo* to estimate the expected winnings based on 10,000 runs. Does the result agree with your theoretical expected value?

Solution: The probability of losing one dollar is 1/2. The probability of "nothing" is 1/3. The probability of winning nine dollars is 1/6.

The set up

```
winnings=c(-1, 0, 9)
probs=c(1/2, 1/3, 1/6)
```

The theoretical expectation is

```
(mu=sum(winnings*probs))
```

```
## [1] 1
```

Using *Monte Carlo*:

```
n.sims=10^4
(mu.mc=mean(sample(winnings, n.sims, replace=TRUE, prob=probs)))
```

```
## [1] 0.9858
```

Problem #3: The binomial distribution

Consider a random variable X which has the binomial distribution with the number of trials equal to 50 and the probability of success in every trial equal to 2/5.

What is the expected value of this random variable?

Use *Monte Carlo* to estimate the expected value based on 10,000 runs. Does the result agree with your theoretical expected value?

Solution: The parameters of X are

```
trials=50
succ.prob=2/5
```

The theoretical expectation is

```
(mu=trials*succ.prob)
```

```
## [1] 20
```

The *Monte Carlo* estimate is

```r
n.sims=10^4

(mu.mc=mean(rbinom(n.sims, trials, succ.prob)))
```

```
## [1] 20.0327
```